

-



- [Core Concepts](#)
- [Models](#)
- Model Training

On this page

Model Training

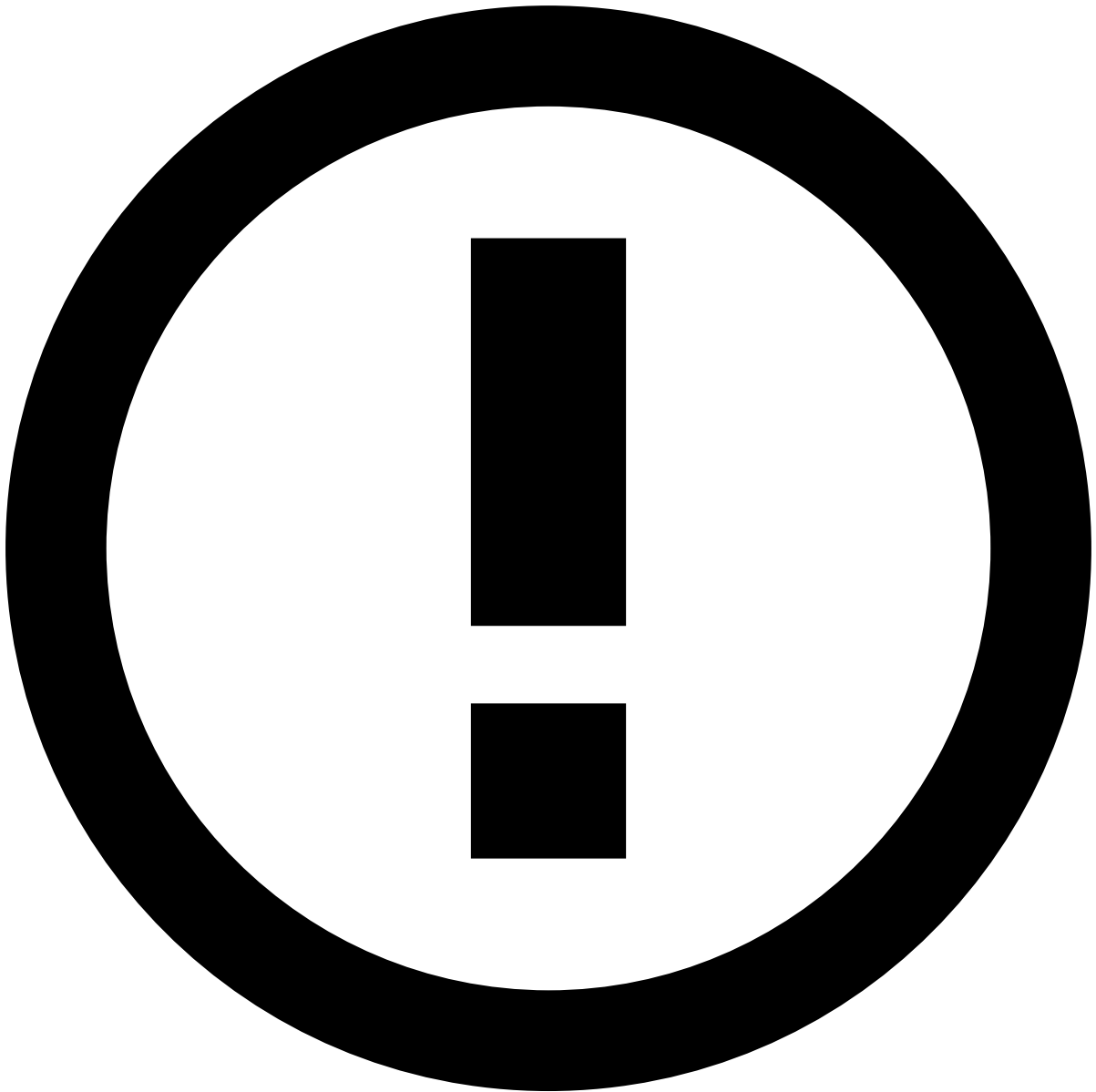
Was this page helpful? 

Training a model means running historical data through a machine learning algorithm to produce a model. You use the trained model in workflows to predict the value of a specified **target variable** in live data.

As part of the [data science loop](#), it is common to try different model training algorithms or keep making adjustments to model parameters and [feature plans](#), and repeat training until you reach a satisfactory performance.

The following sections give you more details on how models are trained in Pulse.

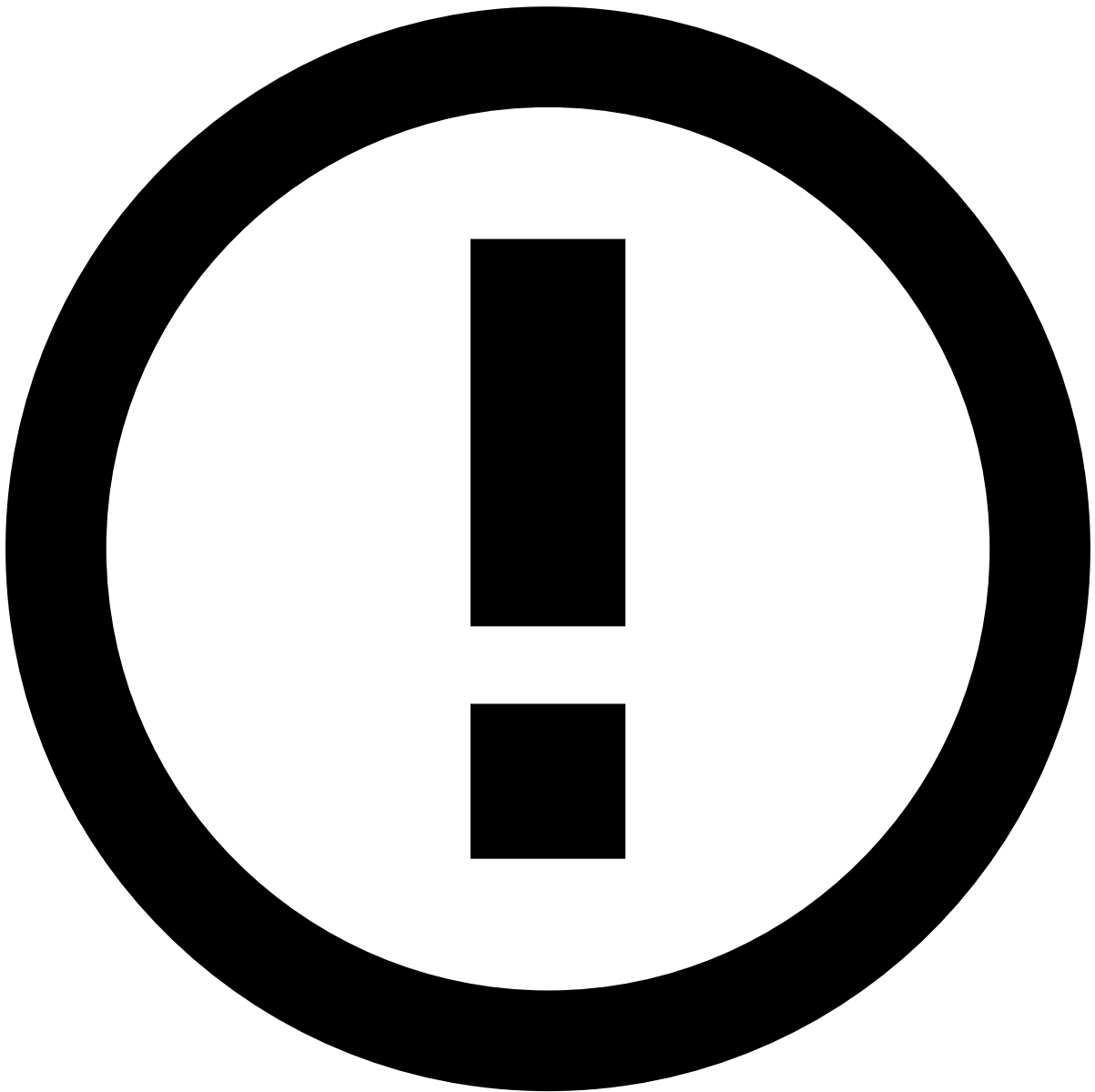
The event's journey during the model training



Naming conventions

When a transaction is flowing in the workflow, it is called an **event**. When an event is enriched with features, it is known as an **instance**. The instance does not traverse the workflow; it is computed using the event data and then used for training (offline data science) or scoring (runtime).

When Pulse loads a training event, it can be filtered before reaching a feature plan. If it reaches the feature plan, an instance with the computed real-time and scheduled metrics is generated. It also includes data transformations and enriched data, in addition to some of the raw features from the event. The instance can then be filtered again before generating the final data source, which Pulse will use to train the model.



Wasted computation?

It might seem odd to compute metrics for events that can later be dropped, but it actually has a simple use case. For instance, when training a model for fraud prevention in card-not-present transactions, it might be important to consider the number of card-present transactions. In this case, you could drop the card-present transactions using a [filter operator](#) before generating the plan output and after computing the count metric.

Omnichannel models

Note that in some cases you might want to train models using data from several data sources for [channels](#). For example, you might be scoring debit card transactions (main data source) and leveraging information about internet banking events (secondary data source). This can be achieved by executing an [omnichannel plan](#) on the input data sources. The output of that plan can then be used for model training.

Supervised and unsupervised machine learning

Pulse allows two different machine learning approaches:

- Training a model through **supervised machine learning**, where the purpose of the model is to predict the label of a target variable. To train a classification model, the data used for the training needs to be labeled.
- Training a model with **unsupervised machine learning**. In this case, a target variable is not used for training.

Target variable

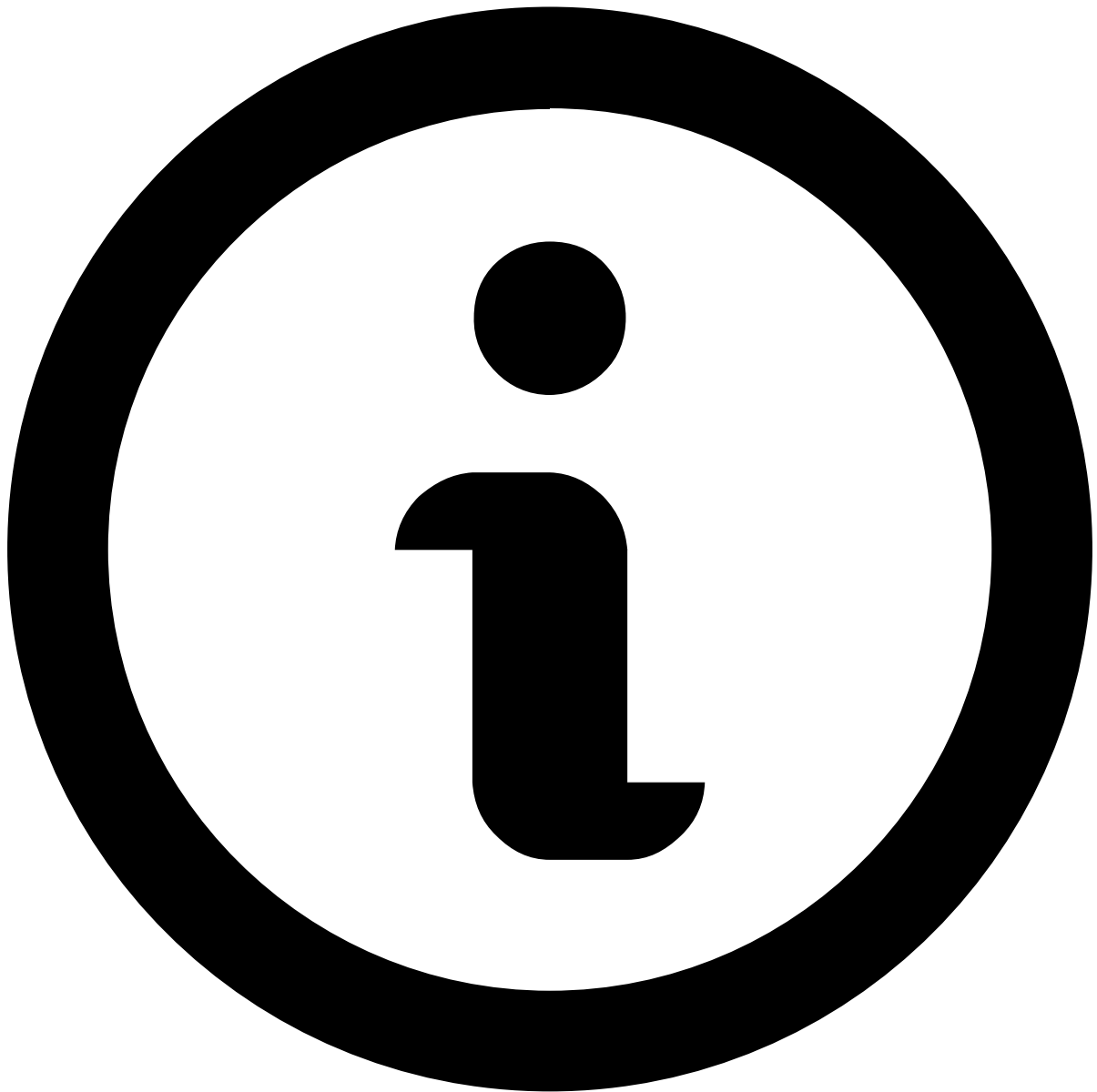
The purpose of a classification model is to predict the label of the target variable. Although Pulse's main use case is fraud detection, you can run your [data science loop](#) and train your models to predict the value for any [categorical](#) target variable, not just fraud/not-fraud. This way you can solve the following classifications problems, during both the data science stage and in Production:

- Binary classification, for example fraud/not fraud.
- Multiclass classification for target variables with more than two categories.

As mentioned before, unsupervised machine learning models don't require a target variable for training.

External models

Instead of creating and training your model in Pulse, you can [import a model](#) that you trained in an external system (using DataRobot, H2O, scikit-learn, the caret package in R, or custom code written in Python or R).



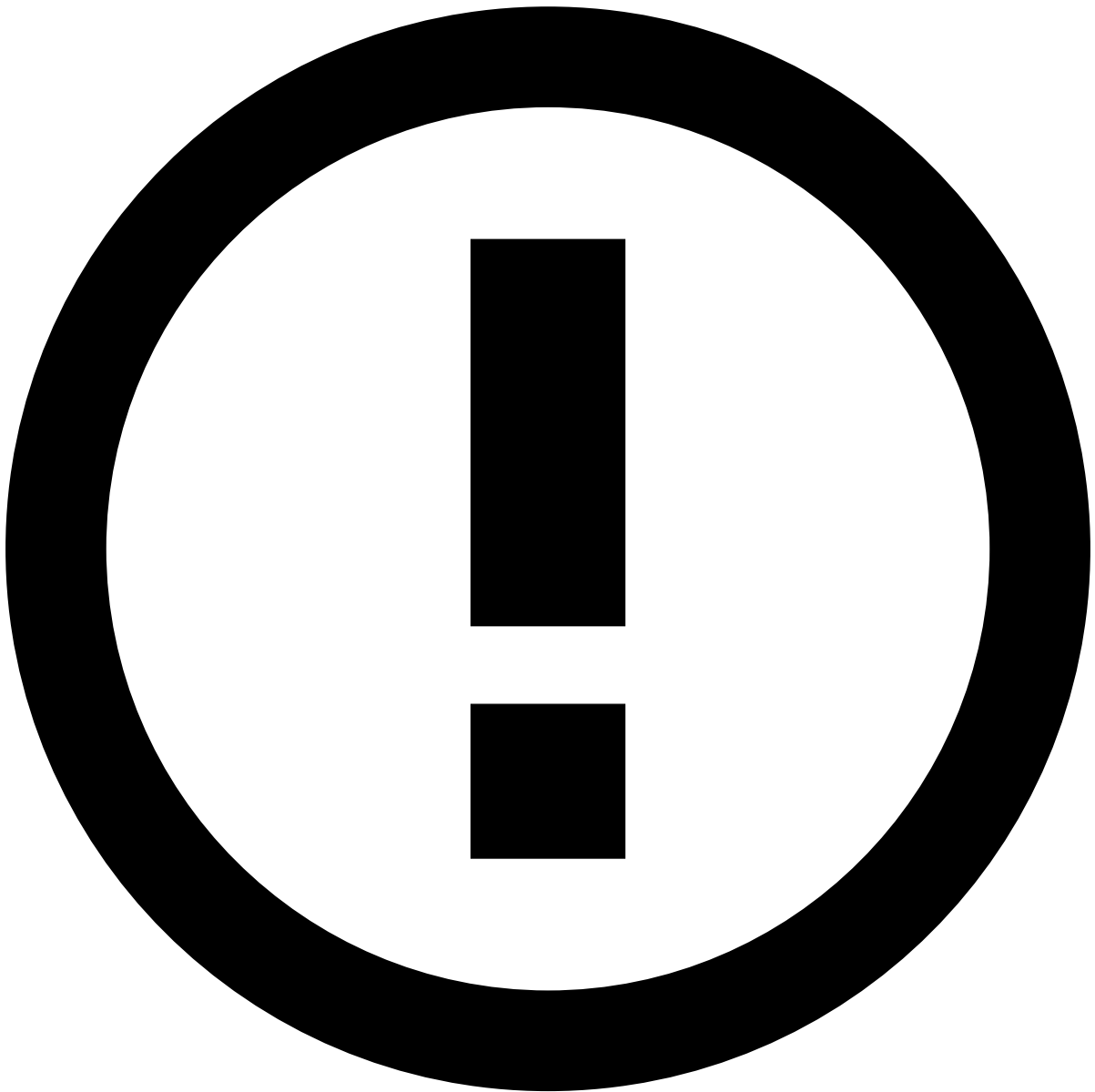
Known limitation - DataRobot

DataRobot allows training models based on string fields, but Pulse doesn't support that. To achieve the same in Pulse, convert the fields that have string type to categorical fields, before training a model based on that data.

Validation method

When training a classification model, it is important to validate the model performance with an independent set of data that was not used to train the model. Pulse automates the task of validation and allows you to select from the following options:

- Validation using the split percentage method for a given ratio
- Cross validation up to 20 folds
- Validation against a second data source
- No validation



Memory consumption of training models

Pulse executes model training jobs as a distributed job. It's recommended to monitor and optimize the memory

allocated to distributed jobs, to ensure that they have the necessary resources to run successfully. In case the distributed job encounters memory issues, consider the following:

- During model training, the training data set is fully in memory.
- For the split percentage and cross validation methods, the evaluation data set is also fully in memory.
- Validation against a second data source executes as a separate job. Therefore, it does not impact the memory of the training.

Split percentage 📊

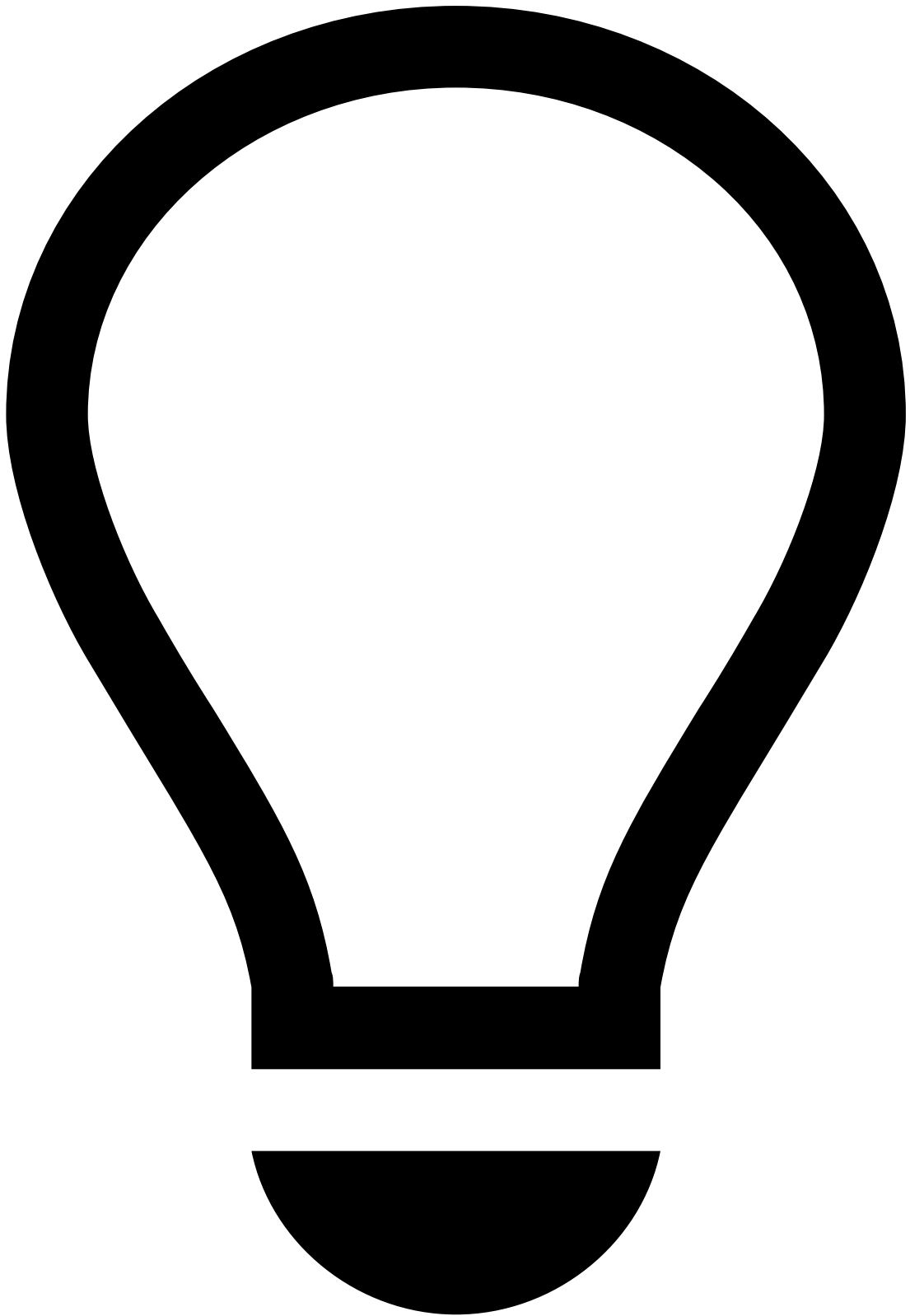
Split percentage of an $X\%$ ratio just uses that amount of data for training and the remaining $(100 - X)\%$ for validation.

The model that is stored, however, is trained with 100% of the data. This validation method is simple and fast.

Although the split is done deterministically, the obtained performance metrics is just a single data point, hence this validation method is not reliable for time-aware datasets. In addition, because we don't know the natural order of the instances of the dataset, if the specific problem being tackled is time-based (e.g. fraud detection), then we want to guarantee that the transactions are ordered in time before doing the split. Otherwise, we may be training the model with "future" patterns and testing with past ones.

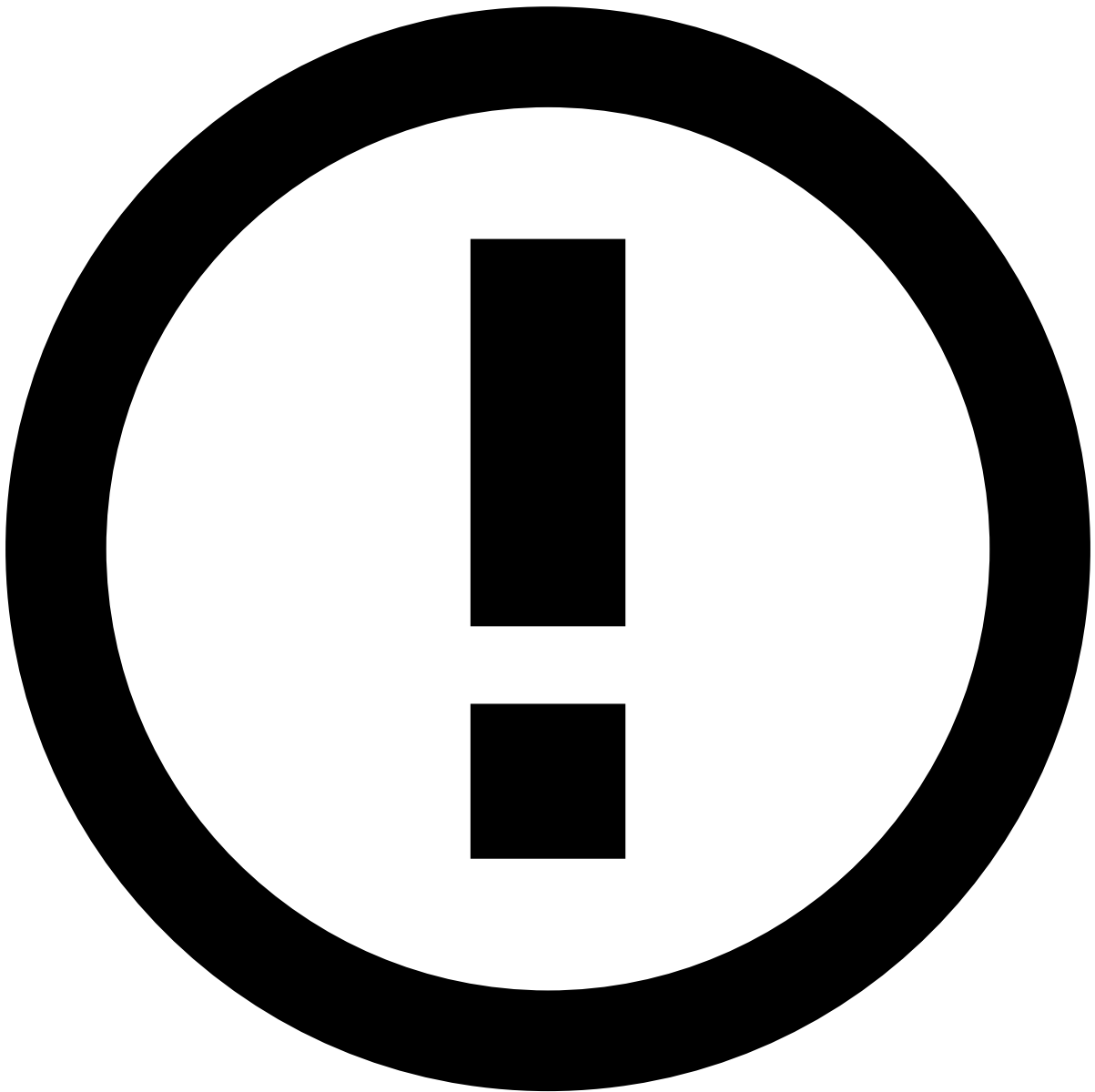
On the other hand, if all hard-to-classify instances are placed in the training dataset, the results will be inflated since it will only need to validate against easier instances. These are all important considerations to take in any type of evaluation in addition to the split percentage.

A default value of 70% ratio is a common practice.



When to use split percentage

You should opt for the split percentage method for quick iterations, especially if your dataset is very large, and for non-time-aware problems.



What makes an instance easy for a binary classifier?

Models attempt to extract patterns from the observed data. With more unique instances, the model is able to better understand what the target concept is, and how to define it (Is this transaction fraudulent? Yes, because ...).

Intuitively, this reasoning only has significance inside the domain of observed data. When scoring an instance that *looks like* fraudulent transactions that were previously observed by the classifier, it can easily tag the instance as fraud.

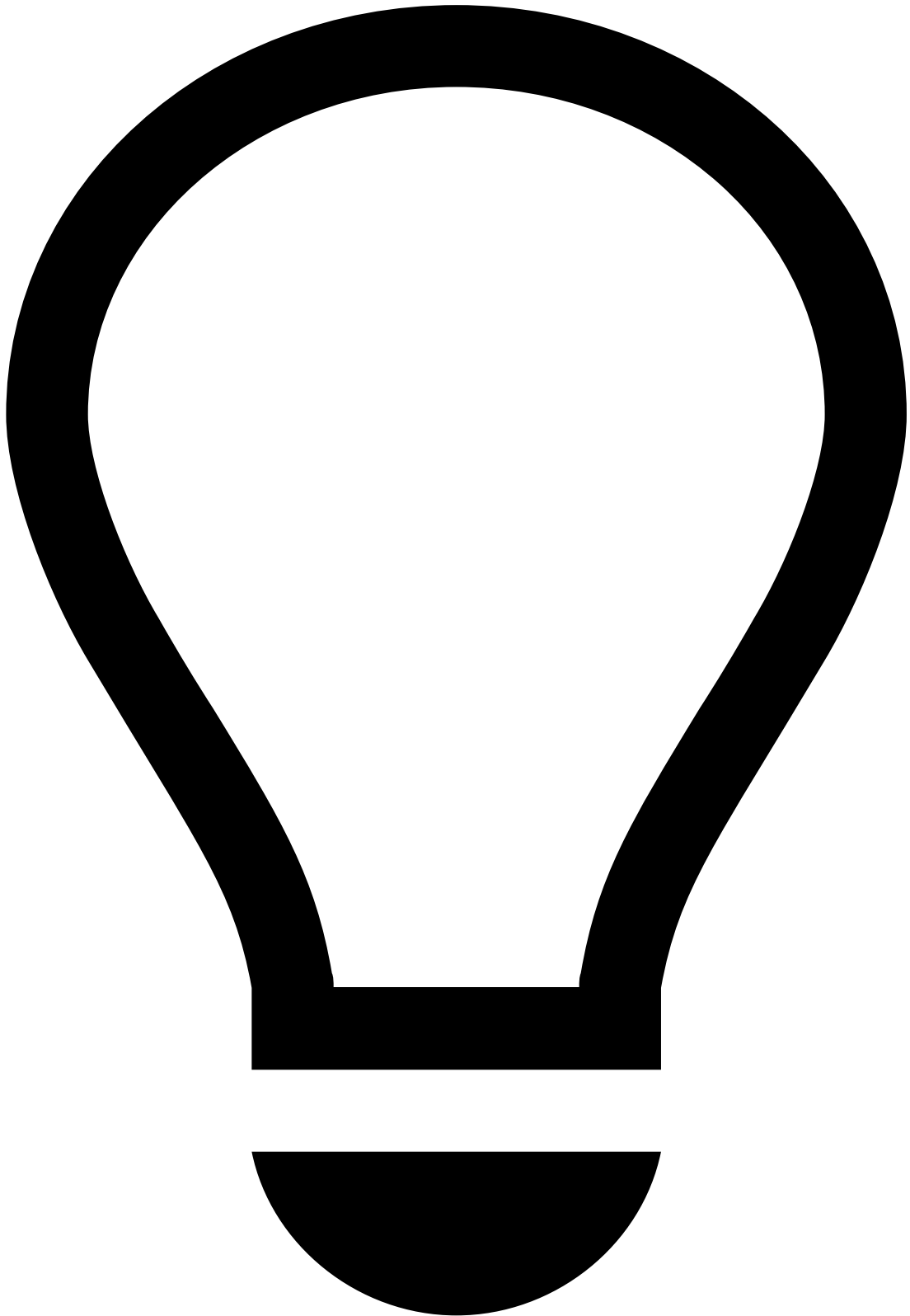
However, when there are no transactions that look like the one requested, or there are a few but they have mixed classes, this decision becomes harder.

Cross validation 🧪

Cross validation is a method that attempts to minimize the variability observed with split percentage. It splits the data into N equally-sized blocks, and uses $N-1$ blocks for training and 1 for validation. The training/validation process is repeated by shifting the blocks until all N blocks have been used for validation. Choosing $N=10$ would yield the following sequence of training and validation runs:

The results are then equally averaged creating a final rundown. The averaging process inherently minimizes the variability of the result. However, the greatest advantage over split percentage is that each instance appears both in the training and in the test dataset (in different runs) thus ensuring that each instance influences the final result.

Nonetheless, cross-validation requires training $N-1$ models, which might be unwieldy for large datasets.



When to use cross validation

You should opt for this method in the final iterations of your [data science loop](#) when micro-optimizing the model definition, or if the training operation is fast enough that doesn't break your work cycle.

You shouldn't use it for time-aware problems.

Validation against a second data source

This validation method allows you to validate the model against a data source of Pulse, other than the training data source.

The data fields selected for the validation must match the fields of the training data set.

Sampling strategy

Sampling strategies are techniques that artificially modify the class distribution of the dataset. This can be used for biasing the model creator into believing that the class distribution is different from the one observed in the dataset. The class distribution is heavily biased in the real world (such is the case with fraud). Class imbalance is a widely known problem that affects many machine learning models. The problem arises when the number of instances from one class is so low that they become mostly irrelevant for training purposes. This generates classifiers that do not have enough sensitivity for the uncommon class (failing to detect fraudulent transactions and instead opting to score every transaction as valid; this has better accuracy compared to misclassifying a few instances of both classes).

When defining the sampling strategy, you can specify the $X\%$ that you would like the dominant class to have. The strategy will transform the dataset in a way that ensures this ratio.

Imagine that your training dataset consists of 99.5% non-fraud transactions and 0.5% fraud transactions, and you wish to train the model with 95% and 5% respectively.

You can achieve the desired ratio with either an undersampling or an oversampling filter both with 95% ratio (because non-fraud is the prevalent class).

- **Oversampling**

Produces more data of the minority class (by copying) to achieve the desired ratio. The instances of the dominant class are kept intact and the dataset grows in size.

- **Undersampling**

Deletes instances from the majority class to meet the ratio. Instances of the minority class are kept intact and the dataset shrinks in size.

Depending on your dataset, you should use a different sampling strategy. On the one hand, oversampling introduces more instances but the information contained in them is the same. This makes the dataset bigger (but not better) and therefore the training process is slower. On the other hand, undersampling loses information that was present in the deleted instances.

Improving models

A model is defined by a set of parameters: some are defined by the user, whereas others are selected based on the training data when applying the algorithm to the data.

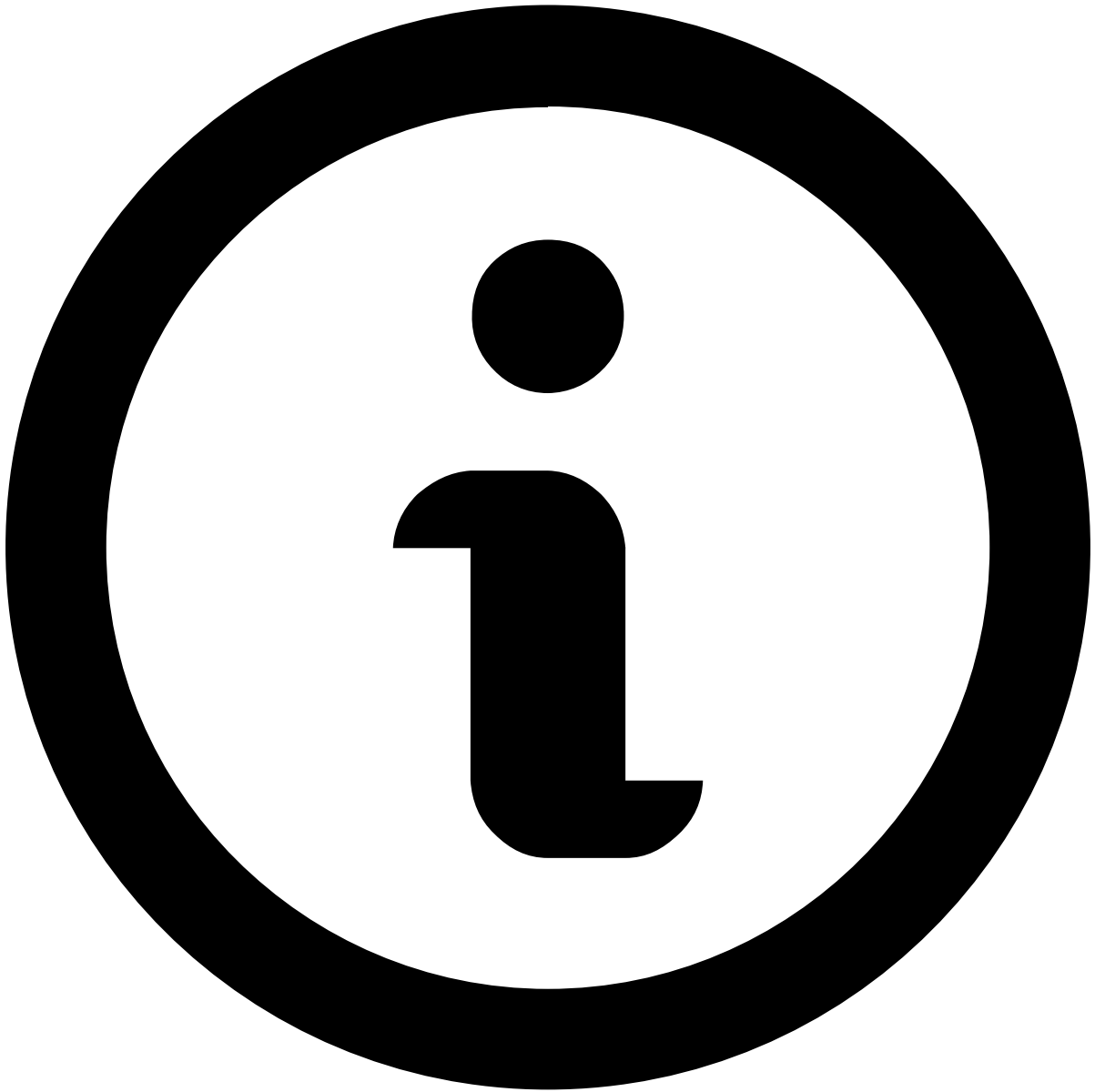
The performance of the model can be improved by:

- Optimising the hyperparameters (parameters that can be selected by the user). This process is named hyperparameter tuning.
- Comparing different machine learning algorithms. For example, XGBoost vs Random Forest vs Deep Learning.
- Changing the training dataset. For example, to a more recent dataset.

Adjust the feature set that is being used to train the model. Some patterns might not be captured by the current feature set. Bear in mind that for a fair and insightful comparison you should be assessing the performance in the same data set.

Configuring white-box explanations

During model training, users can configure the **white-box explanations**, which consists in the calculation of the features that led to a transaction's score. This calculation occurs in runtime after adding the **trained model** to the workflow.



note

Although white-box explanations are configured in the Pulse UI, their output is only visible in Case Manager.

For more information on the parameters that must be configured in this field, visit the [Training a Model](#) page.

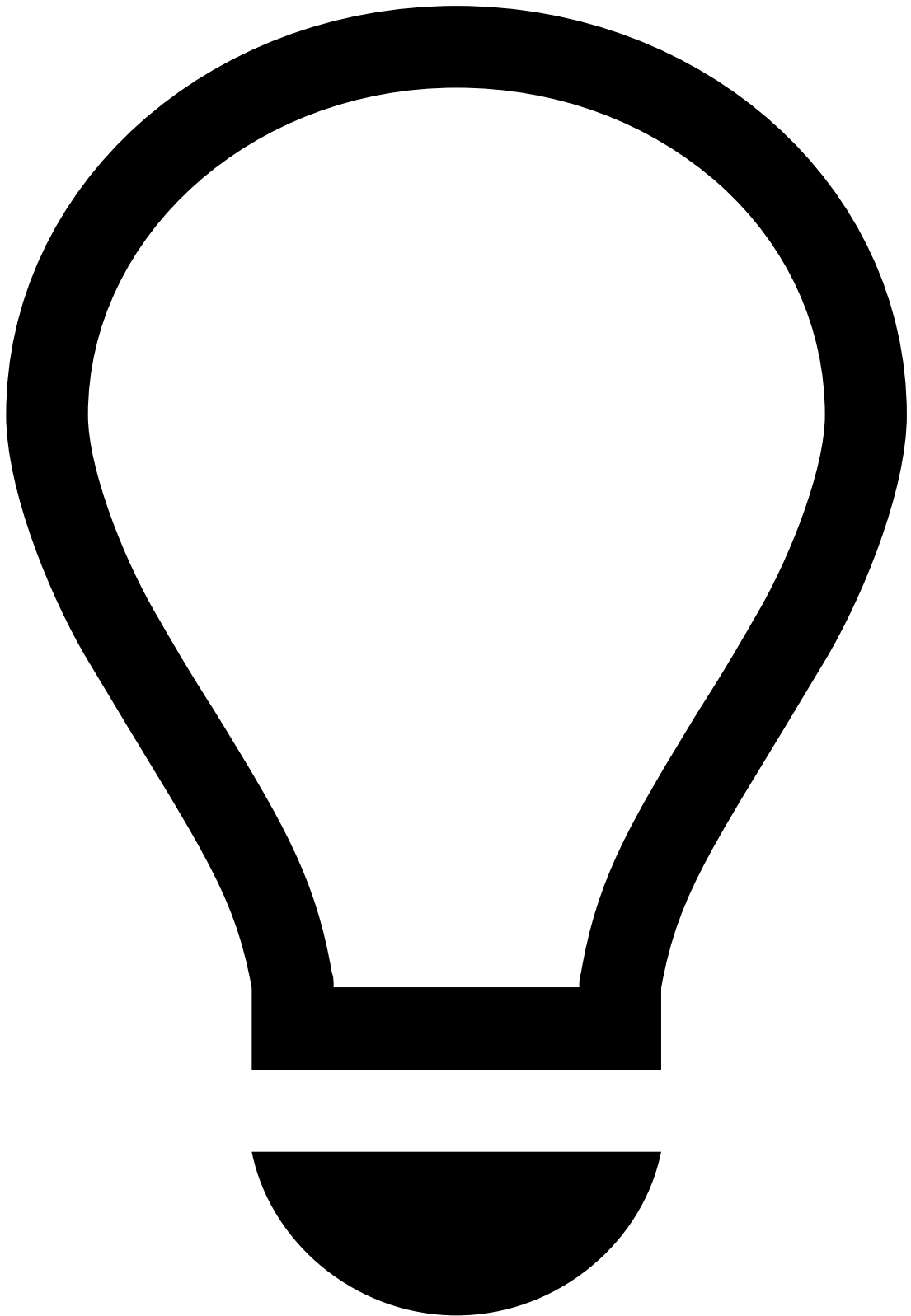
For more information on this subject, read the [White-Box Explanations](#) page.

Immutability

Since a model is immutable by definition, once you train it with a specific dataset and machine learning parameterization, it will always yield the same results for a given test dataset. If you change the parameters or the training dataset, you are effectively generating a different model.

In alignment with this, when you create a model in Pulse, the configuration you used to train it cannot be changed afterward. Once you configure algorithm parameters and the input data source when [training a model](#), these are permanently part of what distinguishes a model iteration from the previous one in your project.

So if you change any model dependency such as the data source, which is also immutable, you are creating a new iteration in your [data science loop](#). In addition, immutability allows Pulse to isolate the model during the data science phase from the same model when deployed to Production. This way you can safely correct and improve your models without affecting your Production workflow.



Deploying improved models

If you need to deploy corrections and improvements to your current model, you can simply run the training procedure and generate a new model. Then configure the scoring service in your workflow with the new model.

Nevertheless, Pulse still allows you to effectively manage your model project and the performance of multiple model iterations. [Model evaluations](#) are what exposes results and not the model itself. A model evaluation always produces the same results over a given model, reinforcing the need for different models for each iteration in your project.

Learn more

Check the following pages in the User Guide to learn how to put models into practice:

- [Training a Model](#)
- [Importing an External Model](#)
- [Model Evaluation](#)